VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

Síťové aplikace a správa sítí – projekt

Filtrující DNS resolver

15. listopadu 2025

Peter Huňady (xhunadp00)

Obsah

1 Úvod **2**
- 1.1 Cieľ projektu . . . 2
- 1.2 Téoria . . . 2
  - 1.2.1 DNS . . . 2
  - 1.2.2 DNS hlavičková sekcia . . . 2
  - 1.2.3 DNS otázková sekcia . . . 2

2 Implementácia **3**
- 2.1 Štruktúra projektu a popis . . . 3
  - 2.1.1 dns.cpp . . . 3
  - 2.1.2 src/parameters.cpp . . . 4
  - 2.1.3 src/filter.cpp . . . 4
  - 2.1.4 src/sockets.cpp . . . 4
  - 2.1.5 src/dnsPacket.cpp . . . 4
  - 2.1.6 Zvyšné odovzdané súbory . . . 4
- 2.2 Chybové kódy a chybové hlášky . . . 4

3 Spustenie a preklad **6**
- 3.1 Kompilácia projektu . . . 6
- 3.2 Spustenie testov . . . 6
- 3.3 Vyčistenie projektu . . . 6
- 3.4 Parametre . . . 6
  - 3.4.1 Základné parametre . . . 6
  - 3.4.2 Rozšírenia (parametre) . . . 6
- 3.5 Príklad použitia . . . 6
  - 3.5.1 Výpis nápovedy . . . 6
  - 3.5.2 Príklad spustenia filtrovania . . . 6
  - 3.5.3 Spustenie programu s parametrom verbose . . . 7

4 Testovanie **8**
- 4.1 Manuálne testovanie . . . 8
  - 4.1.1 1. príklad manuálneho testovania . . . 8
  - 4.1.2 2. príklad manuálneho testovania . . . 8
  - 4.1.3 3. príklad manuálneho testovania . . . 9
  - 4.1.4 4. príklad manuálneho testovania . . . 9
- 4.2 Automatizované testovanie . . . 9

5 Rozšírenia **11**
- 5.1 Verbose . . . 11
- 5.2 Help . . . 11
- 5.3 Retransmission a timeout . . . 11
- 5.4 Paralelné spracovanie . . . 11

6 Zoznam naštudovanej literatúry použitej pri implementácii **12**

7 Zoznam použitej literatúry **13**

1 Úvod

1.1 Cieľ projektu

Cieľom tohto projektu bolo vytvoriť filtrujúci DNS resolver v jazyku C alebo C++, ktorý bude prijímať DNS dotazy typu A pomocou UDP. Na vstupe programu je zadaný DNS server, port (predvolený je 53) a názov filtrovacieho súboru. Vo vnútri filtrovacieho súboru sú domény, ktoré sú nežiaduce. Program po prijatí dotazu skontroluje, či sa doménové meno zhoduje s nejakou nežiaducou doménou zo súboru. Ak áno, program pošle DNS správu REFUSED klientovi. Ak nie, dotaz sa preposiela na zadaný DNS server, počká na odpoveď a tú vráti v nezmenenej podobe klientovi.

1.2 Téoria

1.2.1 DNS

DNS alebo systém doménových mien prekladá ľudsky čitateľné názvy domén na strojovo čitateľné IP adresy.[1] To znamená, že užívatelia si nemusia pamätať IP adresy jednotlivých webstránok, napríklad *157.240.30.35* (IPv4 pre www.facebook.com) alebo *2a03:2880:f13d:83:face:b00c:0:25d* (IPv6 pre www.facebook.com), DNS preloží doménu za nich.

1.2.2 DNS hlavičková sekcia

Každá DNS správa má 12 bajtovú hlavičku, ktorá určuje typ správy, stav a počty záznamov v ďalších častiach. Hlavička obsahuje **ID** (16 bitov), ktoré značí identifikátor dotazu, **QR** (1 bit), ktorý značí či ide o dotaz (0) alebo odpoveď (1), **OPCODE** (4 bity) značí o aký typ dotazu sa jedná, **AA** (1 bit) zase značí, či sa jedná o auritatívny server. Ďalej **TC** (1 bit) značí, či sa jedná o skrátenú správu kvôli limitu veľkosti správy, **RD** (1 bit) značí, či v dotaze bolo rekurzívne spracovanie vyžadované, **RA** (1 bit) značí, či server podporuje rekurziu, **Z** (3 bity) značí rezervované bity, ktoré musia stále byť 000. **RCODE** (4 bity) značí návratový kód v odpovedi:

- 0 – Bez chyby (NOERROR),
- 1 – Formátová chyba (FORMERR)
- 2 – Chyba servera (SERVFAIL)
- 3 – Názov neexistuje (NXDOMAIN)
- 4 – Nepodporovaný typ dotazu (NOTIMP)
- 5 – Odmietnuté (REFUSED)

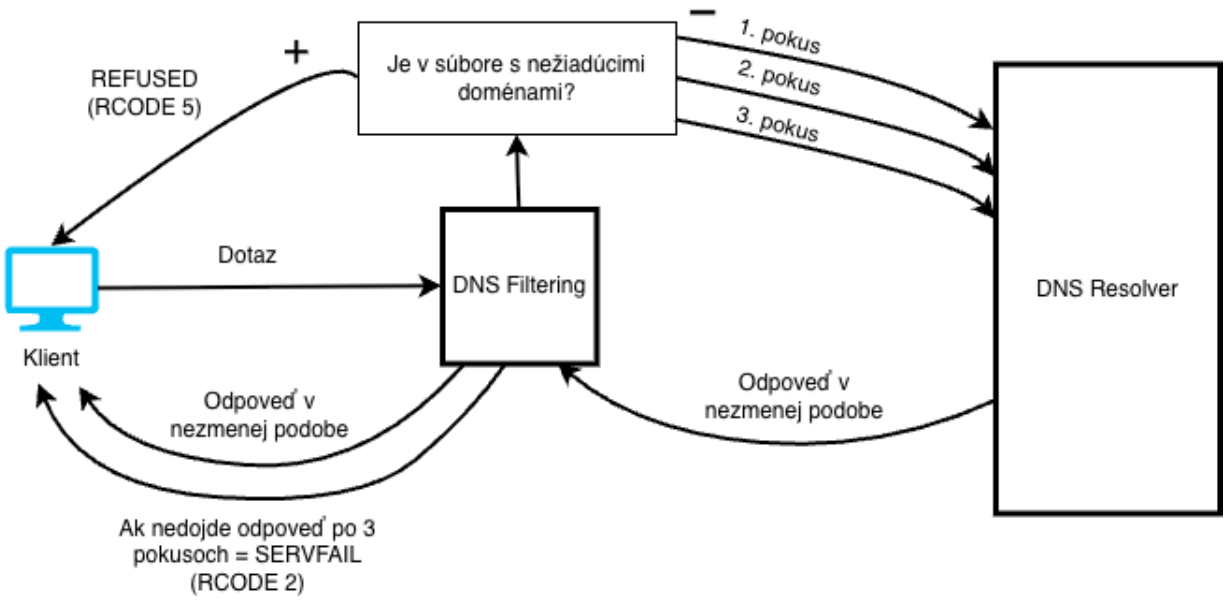
Ako posledné máme všetky 16 bitové časti. **QDCOUNT**, ktorý značí počet záznamov v sekcii *Question*, **ANCOUNT** počet záznamov v *Answer*, **NSCOUNT** počet záznamov v *Authority* a **ARCOUNT** počet záznamov v *Additional*.[2]

1.2.3 DNS otázková sekcia

Obsahuje informáciu, čo klient od DNS servera požaduje. Počet záznamov v tejto časti je určený poľom **QDCOUNT** v hlavičke. Každý záznam v otázkovej sekcii má **QNAME**, čo je vlastne doménové meno v postupnosti jednotlivých labelov. Každý label začína bajtom s dĺžkou a nasleduje daný počet znakov. Potom **QTYPE** (16 bitov), ktorý značí typ záznamu. Napríklad *A* pre IPv4, *AAAA* pre IPv6, *SOA* pre základné informácie o zóne, *NS* pre utoritatívne DNS servery pre danú zónu a podobne. Ako posledné má **QCLASS** (16 bitov), ktorý značí triedu dotazu.[2]

2 Implementácia

Ako prvé program spracuje parametre, ktoré boli zadané na vstupe (DNS server, port, súbor). Ako ďalšie sa načítavajú a spracovávajú domény zo súboru s nežiaducími doménami. Potom program vytvorí UDP socket na zadanom porte a v hlavnom cykle prijíma DNS dotazy od klientov. Pre každý dotaz skontroluje doménu, či sa nachádza v súbore s nežiaducími doménami. Ak je zakázaná pošle sa klientovi odpoveď so stavom REFUSED. Ak je povolená, vytvorí sa nové vlákno, ktoré preposiela dotaz na DNS server. Každé vlákno má vlastný socket pre komunikáciu s DNS serverom, čím zaručíme paralelné spracovávanie dotazov bez blokovania. Socket má nastavený timeout na 2 sekundy, v prípade nedoručenej odpovede sa opakuje proces ešte 2-krát (celkovo 3-krát retransmission). Ak všetky pokusy zlyhajú, klient dostane správu so stavom SERVFAIL. V opačnom prípade sa prepošle správa od DNS servera klientovi v nezmenej podobe.



Obrázek 1: Implementácia

2.1 Štruktúra projektu a popis

Projekt a jeho zdrojový kód je rozdelený do viacerých modulov tak, aby bola oddelená práca so socketmi, spracovávanie parametrov, čítanie zo súboru s nežiaducimi doménami a spracovávanie DNS správ.

2.1.1 dns.cpp

Hlavný súbor programu obsahujúci `main()` a logiku paralelného spracovania. Po spustení načíta parametre a filter, vytvorí socket pre odpočúvanie a v hlavnom cykle prijíma DNS dotazy od klientov. Pre každý dotaz, ktorý prejde filtrom sa vytvorí nové vlákno pomocou *pthread*, ktoré vo funkcii `handleQuery()` preposiela dotaz na DNS server a čaká na odpoveď. V tejto funkcii je implementovaný retransmission s 3 pokusmi s nastaveným timeoutom (2 sekundy) pri každom pokuse. Pri zlyhaní posiela SERVFAIL (RCODE 2) klientovi. Na synchronizáciu odosielania odpovedí klientom sa používa mutex, a pred ukončením program počká na dokončenie všetkých vlákien pomocou *pthread_join()*.

2.1.2 src/parameters.cpp

Pomocou funkcie `parseParameters()` sa spracovávajú parametre zo vstupu a kontroluje sa ich validita. Napríklad, či boli zadané validné parametre (-s, -f, -p, -v, -h/–help), či náhodou parameter nebol použitý viackrát, či zadaný port je v správnom intervale (0-65535) a podobne. Ak prebehla validácia úspešne, parametre sa uložia do štruktúry `Parameters`.

2.1.3 src/filter.cpp

Pomocou funkcie `getAllDomains()` sa načítava zoznam nežiaducich domén zo zadaného súboru. Každý riadok súboru predstavuje jednu doménu, pričom sa ignorujú komentáre začínajúce # a prázdne riadky. Všetky domény sa konvertujú na lowercase a trimujú sa medzery, výsledný zoznam sa vracia ako vector reťazcov.

2.1.4 src/sockets.cpp

Obsahuje dve funkcie pre vytváranie UDP socketov. Funkcia `listeningSocket()` vytvorí socket pre počúvanie DNS dotazov od klientov s nastaveným timeoutom (1 sekunda) pre graceful shutdown. Funkcia `resolvingSocket()` vytvorí socket pre komunikáciu so zadaným DNS serverom pomocou *getaddrinfo()* a nastaví receive timeout na 2 sekundy.

2.1.5 src/dnsPacket.cpp

Obsahuje funkcie pre prácu s DNS paketmi a ich validáciu a taktiež 2 štruktúry `dnsHeader` a `dnsQuestion` pre DNS hlavičkovú a otázkovú sekciu. Funkcia `checkDnsPacket()` kontroluje, či je DNS dotaz validný (správna veľkosť, hlavička, dotaz typu A...) a spracováva z neho doménové meno. Potom sa táto doména porovnáva so zoznamom nežiaducich domén, pričom sa kontrolujú aj subdomény (napríklad www.facebook.com je subdoménou facebook.com). Funkcia `sendErrorPacket()` vytvára chybové DNS odpovede (REFUSED – RCODE 5, NOTIMP – RCODE 4...) pomocou bitových štruktúr pre DNS hlavičkovú sekciu.

2.1.6 Zvyšné odovzdané súbory

- MAKEFILE – Skript na skompilovanie a čistenie projektu a spustenie testov.
- README – Základné informácie o projekte.
- LICENSE – Licencia projektu.
- manual.pdf – Dokumentácia k projektu (tento súbor).
- tests/tests.sh – Bash script s testami k projektu.

2.2 Chybové kódy a chybové hlášky

- **0** – Program skončil úspešne.
- **1** – *"Error: Invalid parameter."*
  Nesprávny parameter na vstupe.
- **2** – *"Error: Missing server name."*
  Chýbajúce meno DNS servera na vstupe.
- **3** – *"Error: Invalid port number. Port must be in range 0-65535."*
  Zadané číslo portu je mimo interval 0-65535.

- **4** – *"Error: Missing name of filter file."*
  Chýbajúce meno súboru s nežiaducimi doménami.
- **5** – *"Error: Parameter server is required."*
  Parameter *-s* pre nastavenie DNS servera je povinný.
- **6** – *"Error: Parameter file name is required."*
  Parameter *-f* pre nastavenie súboru s nežiaducimi doménami je povinný.
- **7** – *"Error: Server parameter (-s) already set."/ "Error: Port parameter (-p) already set."/ "Error: Filter file parameter (-f) already set."*
  Jeden z parametrov (*-s*, *-f*, *-p*) bol zadaný viackrát.
- **8** – *"Error: Opening a file failed."*
  Došlo k chybe pri otváraní súboru s nežiaducimi doménami.
- **9** – *"Error: Empty filter file."*
  Súbor s nežiaducími doménami je prázdny. Keďže úlohou projektu je implementovať filtrujúci DNS resolver a nie DNS forwarder, vyhodíme to ako chybu.
- **10** – *"Error: Listening socket failure."*
  Došlo k chybe pri vytváraní odpočúvacieho socketu.
- **11** – *"Error: Listening socket bind failure."*
  Došlo k chybe pri binde v odpočúvacom sockete.
- **12** – *"Error: Failed to set listening socket timeout."*
  Došlo k chybe pri nastavovaní timeoutu na odpočúvacom sockete.
- **13** – *"Error: Invalid DNS server name."*
  Zadaný DNS server na vstupe neexistuje.
- **14** – *"Error: Failed to connect to server."*
  Nepodarilo sa pripojiť na zadaný DNS server.
- **15** – *"Error: Failed to set resolving socket timeout."*
  Došlo k chybe pri nastavovaní timeoutu na sockete pre posielanie dotazov.

3 Spustenie a preklad

3.1 Kompilácia projektu

```
$ make
```

3.2 Spustenie testov

```
$ make test
```

3.3 Vyčistenie projektu

```
$ make clean
```

3.4 Parametre

3.4.1 Základné parametre

- **-s** – IP adresa alebo doménové meno DNS serveru, kam sa zašle dotaz.
- **-f** – Meno súboru, ktorý obsahuje nežiaduce domény.
- **-p** – Číslo portu na ktorom bude socket očakávať dotazy. Predvolený port je 53.

3.4.2 Rozšírenia (parametre)

- **-h/–help** – Výpis nápovedy k použitiu programu.
- **-v** – Výpis toho, čo sa deje pri preklade a v pozadí programu.

3.5 Príklad použitia

3.5.1 Výpis nápovedy

```
$ ./dns
```

```
$ -h
```

```
$ --help
```

3.5.2 Príklad spustenia filtrovania

```
$ ./dns -s 8.8.8.8 -p 10543 -f test.txt
```

```
$ ./dns -s 8.8.8.8 -f test.txt
```

Predvolený port bude 53

3.5.3 Spustenie programu s parametrom verbose

```
$ ./dns -s 8.8.8.8 -p 10543 -f test.txt -v
```

Výpis:

```
Server: 8.8.8.8
Port: 10543
Filter file: test.txt

Paramaters successfully parsed.
Trying to open filter file...
Filter file opened succesfully.
Loading domains from filter file...
Domains successfully loaded from filter file.

Creating listening socket...
Listening socket succesfully created.

Creating resolving socket...
Resolving socket succesfully created.

Starting communication...
```

Ďalej sa budú vypisovať informácie pri prijímaní odpovedí:

```
Received DNS query, checking packet...
Parsing domain name from query...
Query: facebook.com
DNS packet is valid.
Checking if 'facebook.com' is in filter list...
Domain 'facebook.com' is NOT in filter list - allowing query.

Query for 'facebook.com' accepted, forwarding to DNS server...

Query for 'facebook.com': Successfully forwarded response to client.
```

4 Testovanie

Tento program som testoval dvomi spôsobmi:

- 1. Manuálne – pomocou príkazu dig a programu Wireshark.
- 2. Automatizovane – script v bash

4.1 Manuálne testovanie

Majme na vstupe súbor test.txt s následujúcimi nežiaducimi doménami:

```
www.google.com
facebook.com

#youtube.com
netflix.com
```

Teraz zapneme náš filtrujúci DNS resolver:

```
$ ./dns -s 8.8.8.8 -p 10543 -f test.txt -v
```

4.1.1 1. príklad manuálneho testovania

V druhom termináli vytvoríme dotaz pomocou príkazu dig:

```
$ dig @127.0.0.1 -p 10543 facebook.com
```

Vo Wiresharku môžeme vidieť, že dotaz na doménu bola zablokovaný (REFUSED), pretože sa nachádzala v súbore s nežiaducimi parametrami:

Time	Source	Destination	Protocol	Length	Info
23.636210	127.0.0.1	127.0.0.1	DNS	73	Standard query 0x32a2 A facebook.com OPT
23.636343	127.0.0.1	127.0.0.1	DNS	73	Standard query response 0x32a2 Refused A facebook.com OPT

Obrázek 2: Wireshark 1. test

4.1.2 2. príklad manuálneho testovania

V druhom termináli vytvoríme dotaz pomocou príkazu dig:

```
$ dig @127.0.0.1 -p 10543 youtube.com
```

Vo Wiresharku môžeme vidieť, že dotaz na doménu bol povolený (NOERROR), pretože aj keď sa nachádzala v súbore s nežiaducimi doménami, bola zakomentovaná pomocou #:

Time	Source	Destination	Protocol	Length	Info
17.038073	127.0.0.1	127.0.0.1	DNS	72	Standard query 0xf596 A youtube.com OPT
17.089824	127.0.0.1	127.0.0.1	DNS	88	Standard query response 0xf596 A youtube.com A 142.251.36.110 OPT

Obrázek 3: Wireshark 2. test

4.1.3 3. príklad manuálneho testovania

V druhom termináli vytvoríme dotaz pomocou príkazu dig:

```
$ dig @127.0.0.1 -p 10543 google.com
```

Vo Wiresharku môžeme vidieť, že doména bola povolená (NOERROR), pretože v súbore s nežiaducimi doménami sa nachádzala iba jej poddoména www.google.com:

Time	Source	Destination	Protocol	Length	Info
32.408242	127.0.0.1	127.0.0.1	DNS	71	Standard query 0x78ac A google.com OPT
32.426230	127.0.0.1	127.0.0.1	DNS	87	Standard query response 0x78ac A google.com A 142.251.36.142 OPT

Obrázek 4: Wireshark 3. test

4.1.4 4. príklad manuálneho testovania

V druhom termináli vytvoríme viac dotazov pomocou príkazu dig:

```
$ dig @127.0.0.1 -p 10543 facebook.com &
dig @127.0.0.1 -p 10543 youtube.com &
dig @127.0.0.1 -p 10543 netflix.com &
dig @127.0.0.1 -p 10543 vut.cz SOA &
dig @127.0.0.1 -p 10543 randomnonexistingdomain.com
```

Vo Wiresharku môžeme vidieť, že dotazy na *facebook.com* a *netflix.com* boli zablokované (REFUSED), pretože sa nachádzali v súbore s nežiaducimi doménami. Dotaz na *youtube.com* bola povolený (NOERROR), dotaz na *vut.cz* bol vykonaný s typom SOA a dostali sme odpoveď so stavom NOTIMP, pretože typ dotazu bol SOA a nie A. Ako posledné, doména *randomnonexistingdomain.com* bola poslaná na DNS sever a dostali sme odpoveď so stavom NXDOMAIN, pretože taký názov domény neexistuje a to sme preposlali naspäť klientovi v nezmenej podobei.

Time	Source	Destination	Protocol	Length	Info
0.638514	127.0.0.1	127.0.0.1	DNS	73	Standard query 0x5e54 A facebook.com OPT
0.638523	127.0.0.1	127.0.0.1	DNS	72	Standard query 0xc9f0 A netflix.com OPT
0.638685	127.0.0.1	127.0.0.1	DNS	73	Standard query response 0x5e54 Refused A facebook.com OPT
0.638703	127.0.0.1	127.0.0.1	DNS	72	Standard query response 0xc9f0 Refused A netflix.com OPT
0.638720	127.0.0.1	127.0.0.1	DNS	88	Standard query 0x0a94 A randomnonexistingdomain.com OPT
0.638724	127.0.0.1	127.0.0.1	DNS	67	Standard query 0x261b SOA vut.cz OPT
0.638767	127.0.0.1	127.0.0.1	DNS	72	Standard query 0xba39 A youtube.com OPT
0.638821	127.0.0.1	127.0.0.1	DNS	67	Standard query response 0x261b Not implemented SOA vut.cz OPT
0.678383	127.0.0.1	127.0.0.1	DNS	88	Standard query response 0xba39 A youtube.com A 142.251.36.110 OPT
0.681456	127.0.0.1	127.0.0.1	DNS	161	Standard query response 0x0a94 No such name A randomnonexistingdomain.com

Obrázek 5: Wireshark 4. test

4.2 Automatizované testovanie

Automatizované testovanie bolo vykonávané pomocou scriptu bash v súbore tests/tests.sh. Tento script vytvára testovacie filtrovacie súbory s nežiaducimi doménami a spúšťa rôzne testy. Spolu týchto testov je 51, pričom v tomto scripte máme niekoľko typov testov:

- **run_valid_test()** – testy, ktoré by mali úspešne prebehnúť bez predčasného ukončenia programu s návratovým kódom iným než 0.
- **run_parameters_test()** – testy, ktoré testujú rôzne vstupné parametre a či program vracia správne návratové kódy pri chybe.

- **run_dns_filtering_test()** – testy, ktoré testujú s príkazom dig zhody stavu v odpovediach na klientovi.
- **Viacero dotazov naraz** – V tomto teste kontrolujeme, či dokáže program spracovať viacero dotazov naraz.
- **Neexistujúci DNS server** – V tomto teste kontrolujeme, či dojde klientovi správa so stavom SERVFAIL, ak je zadaný DNS server neexistujúci. Taktiež týmto testom kontrolujeme retransmission (3 pokusy) a každý so zadaným timeoutom (2 sekundy).

Na konci sa vypíše koľko testov prešlo úspešne a koľko neúspešne. Testy spustíme pomocou príkazu:

```
$ make test
```

Projekt sa preloží a následne sa spustia testy:

```
Commented line (fit.vut.cz): PASS
Multiple subdomains (example.admin.org.facebook.com): PASS
Subdomain of filtered subdomain (admin.subdomain.youtube.com): PASS
Similar domain (www.ggoogle.com): PASS
Similar domain (ttwitter.com): PASS
Case-insensitive (TwiTTer.Com): PASS
Case-insensitive (wWW.gOoGlE.cOm): PASS
Case-insensitive (vUT.cZ): PASS
Not implemented AAAA type of querry filtered domain (twitter.com): PASS
Not implemented MX type of querry (vut.cz): PASS
Not implemented CNAME type of querry (github.com): PASS
Not implemented TXT type of querry (google.com): PASS
Not implemented NS type of querry (example.com): PASS
Not implemented SOA type of querry (vut.cz): PASS
Non-existent domain (whuqdhuewfbjhsabuefugbwzu231231.com): PASS

--------------------------------
     Multiple Queries Test
--------------------------------

Multiple queries (10 parallel requests): PASS

--------------------------------
          SERVFAIL Test
--------------------------------

SERVFAIL with non-responsive DNS server (10.0.11.111): PASS

================================
            Summary
================================
PASSED: 51
FAILED: 0
```

Obrázek 6: Automatizované testy

5 Rozšírenia

5.1 Verbose

Výpis toho, čo sa deje pri preklade a v pozadí programu:

```
Server: 8.8.8.8
Port: 10543
Filter file: test.txt

Paramaters successfully parsed.
Trying to open filter file...
Filter file opened succesfully.
Loading domains from filter file...
Domains successfully loaded from filter file.

Creating listening socket...
Listening socket succesfully created.

Creating resolving socket...
Resolving socket succesfully created.

Starting communication...

Received DNS query, checking packet...
Parsing domain name from query...
Query: facebook.com
DNS packet is valid.
Checking if 'facebook.com' is in filter list...

Query for 'facebook.com' BLOCKED: Domain is in filter list (RCODE 5).
```

Obrázek 7: Verbose

5.2 Help

Výpis nápovedy k použitiu programu:

```
Filtering DNS resolver
Usage: dns -s server [-p port] -f filter_file

-s server: IP address or domain name of DNS server.
-p port: The port number on which the program will listen for queries. Default port is 53.
-f filter_file: name of the file contaning unwanted domains
-v: print resolving informations.
-h | --help: print this message
```

Obrázek 8: Help

5.3 Retransmission a timeout

Aj keď to nebolo súčasťou zadania, tento program dokáže pri komunikácii s DNS serverom opakovane odosielať dotaz pri výpadku alebo časovom omeškaní odpovede od DNS servera. Pre každý dotaz je na sockete nastavený časový limit 2 sekundy. Ak v tomto čase nepríde odpoveď, dotaz bude znovu zaslaný, avšak celkový počet zaslaných dotazov bez odpovede od DNS servera môže byť 3. Pokiaľ ani po 3. pokuse nedorazí odpoveď od DNS servera, program vráti klientovi chybovú odpoveď so stavom SERVFAIL, čím signalizuje problém na strane servera alebo siete.

5.4 Paralelné spracovanie

Program dokáže spracovávať viacero DNS dotazov súčasne. Hlavné vlákno prijíma dotazy na počúvajúcom UDP sockete a pre každý dotaz vytvorí nové vlákno, ktoré zabezpečí komunikáciu s DNS serverom. Každé vlákno používa vlastný socket pre komunikáciu s DNS serverom, aby sa predišlo blokovaniu pri čakaní na odpoveď. Pri posielaní odpovedí klientovi sa používa mutex, aby sa naraz neposielalo viac vlákien cez ten istý socket. Tým zaručíme, že každá odpoveď príde správnemu klientovi.

6 Zoznam naštudovanej literatúry použitej pri implementácii

Nasledujúce zdroje som použil pri štúdii problematiky DNS a pri návrhu implementácie filtrujúceho DNS resolveru:

- P. Mockapetris: Domain Names – Implementation and Specification, https://datatracker.ietf.org/doc/html/rfc1035
- Brian "Beej Jorgensen" Hall: Beej's Guide to Network Programming, https://beej.us/guide/bgnet/html/
- PowerCert Animated Videos: How a DNS Server (Domain Name System) works., https://www.youtube.com/watch?v=mpQZVYPuDGU
- Barry Brown: DNS Resolution, Step by Step, https://www.youtube.com/watch?v=3EvjwlQ43_4
- linux.die.net: dig(1) - Linux man page, https://linux.die.net/man/1/dig
- W3Schools: C++ Vectors, https://www.w3schools.com/cpp/cpp_vectors.asp
- Rolf Winter: (#5) Live Coding: DNS resolution in C, https://youtu.be/Pr7A78A-08c?si=lNSTxIftDqIEmjY4
- man7.org: getaddrinfo(3) — Linux manual page, https://man7.org/linux/man-pages/man3/getaddrinfo.3.html
- man7.org: pthreads(7) — Linux manual page, https://man7.org/linux/man-pages/man7/pthreads.7.html
- en.cppreference.com: std::mutex, https://en.cppreference.com/w/cpp/thread/mutex.html
- en.cppreference.com: sig_atomic_t, https://en.cppreference.com/w/c/program/sig_atomic_t.html
- man7.org: sockaddr(3type) — Linux manual page, https://man7.org/linux/man-pages/man3/sockaddr.3type.html
- linux.die.net: ntohs(3) - Linux man page, https://linux.die.net/man/3/ntohs
- The Open Group Base Specifications: recvfrom - receive a message from a socket, https://pubs.opengroup.org/onlinepubs/000095399/functions/recvfrom.html
- man7.org: send(2) — Linux manual page, https://man7.org/linux/man-pages/man2/send.2.html
- The Open Group Base Specifications: sendto - send a message on a socket, https://pubs.opengroup.org/onlinepubs/009604499/functions/sendto.html
- en.cppreference.com: atexit, https://en.cppreference.com/w/c/program/atexit
- man7.org: pthread_join(3) — Linux manual page, https://man7.org/linux/man-pages/man3/pthread_join.3.html

7 Zoznam použitej literatúry

[1] CLOUDFLARE. *What is DNS?* https://www.cloudflare.com/learning/dns/what-is-dns/. 2024. Dostupné online.

[2] MOCKAPETRIS, P. *RFC 1035: Domain Names – Implementation and Specification* https://datatracker.ietf.org/doc/html/rfc1035. 1987. Dostupné online.